

OE-EC803B — Big Data Analysis

Semester VIII • 3L:0T:0P • 3 Credits

Module 3: MapReduce Programming Paradigm & YARN Resource Manager

CGEC | B.Tech ECE (2023–27) | Biraj Sarkar

Contents

1	Anatomy of a MapReduce Job Run	2
1.1	YARN Runtime Job Execution Steps	2
2	Failures in YARN	2
2.1	Task Failure	3
2.2	NodeManager Failure	3
2.3	ResourceManager / ApplicationMaster Failure	3
3	Job Scheduling	3
4	Shuffle and Sort Phase	4
4.1	Map Side Execution Details	4
4.2	Reduce Side Execution Details	4
5	MapReduce Types and Formats	4
5.1	Input Splits and InputFormat	4
6	Advanced MapReduce Features	5
6.1	Counters	5
6.2	Joins in MapReduce	5

1. Anatomy of a MapReduce Job Run

MapReduce execution is divided into MapReduce 1 (Classic MapReduce) and MapReduce 2 (built on YARN).

Classic MapReduce vs. YARN

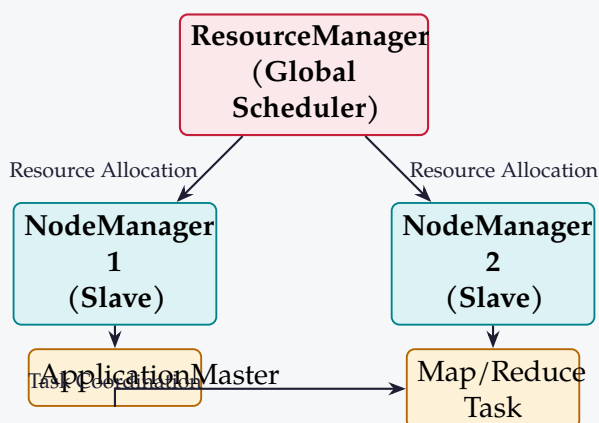
- **MapReduce 1 (Classic):** Job execution was coordinated by a single master process called the **JobTracker**, which scheduled tasks and monitored slaves called **TaskTrackers**.
- **YARN (MapReduce 2):** Divides the resource management and job scheduling functions into separate daemons: a global **ResourceManager** and application-specific **ApplicationMasters**.

1.1 YARN Runtime Job Execution Steps

YARN runs MapReduce jobs using several steps:

1. **Job Submission:** The Client program calls 'submitJob()' on the Job object, requesting a new Application ID from the ResourceManager.
2. **Application Master Launch:** The ResourceManager allocates a **Container** (allocated CPU/RAM limit) on a NodeManager and starts the **ApplicationMaster** process there.
3. **Resource Allocation:** The ApplicationMaster registers with the ResourceManager and requests containers across NodeManagers to host the Map and Reduce tasks.
4. **Task Execution:** The ApplicationMaster contacts the NodeManagers to launch the tasks within the allocated containers. Tasks retrieve data blocks from local HDFS DataNodes, process them, and write outputs locally.

YARN Resource Management Layout



2. Failures in YARN

Distributed systems must tolerate node and task failures gracefully.

2.1 Task Failure

A Map or Reduce task can fail due to runtime Java exceptions or JVM crashes.

- **Detection:** The NodeManager detects that the task JVM exited with a non-zero code and reports the failure back to the ApplicationMaster.
- **Resolution:** The ApplicationMaster reschedules the task on a different NodeManager. It will retry the task up to 4 times (default limit) before marking the entire job as failed.

2.2 NodeManager Failure

- **Detection:** The NodeManager sends heartbeats to the ResourceManager. If heartbeats stop (typically for 10 minutes), the ResourceManager marks the NodeManager as dead.
- **Resolution:** The ResourceManager notifies the active ApplicationMasters. The ApplicationMasters reschedule all tasks (both active and completed map tasks) that were running on the failed NodeManager, since their local map outputs are no longer accessible.

2.3 ResourceManager / ApplicationMaster Failure

- **ApplicationMaster Failure:** The ResourceManager detects the loss of heartbeat from the ApplicationMaster and starts a new container to launch a fresh ApplicationMaster instance.
- **ResourceManager Failure:** This is a single point of failure (SPOF). It is resolved by running active-standby ResourceManagers using ZooKeeper to manage failover and state replication.

3. Job Scheduling

YARN supports three primary job scheduling engines:

1. **FIFO (First-In, First-Out) Scheduler:** Allocates resources strictly based on job submission order. A large, slow job at the head of the queue will block all subsequent small jobs.
2. **Capacity Scheduler:** Allocates distinct organizational queues, where each queue is guaranteed a fixed percentage of cluster capacity (e.g., Queue A gets 40%, Queue B gets 60%). If a queue has free space, it can temporarily lend it to other queues.
3. **Fair Scheduler:** Dynamically balances resources so that all running jobs get an equal share of cluster capacity over time. When a single job runs, it uses 100% of the cluster; when a second job is submitted, YARN re-allocates 50% of the resources to it.

4. Shuffle and Sort Phase

The Shuffle and Sort

The **Shuffle and Sort** is the core transport layer of MapReduce that routes intermediate key-value outputs from mappers to the correct reducers, guaranteeing that all values associated with a specific key are grouped and delivered to the same reducer in sorted order.

The shuffle process runs across the map and reduce stages:

4.1 Map Side Execution Details

1. **Circular Buffer:** The Map task writes outputs to an in-memory circular RAM buffer (default size: 100MB).
2. **Spilling:** When the buffer fill percentage reaches a threshold (default: 80%), a background thread wakes up to spill the data to the local disk.
3. **Partitioning:** Before writing to disk, the thread divides the key-value records into partitions based on the target Reducer ID (using `HashPartitioner` by default: `(key.hashCode() & Integer.MAX_VALUE) % numReducers`).
4. **Sorting:** Within each partition, the keys are sorted in memory using a quicksort algorithm.
5. **Combining:** If a local 'Combiner' (mini-reducer) is defined, it runs on the sorted partition records before writing to disk to reduce data volume.
6. **Merging:** When the Map task finishes, all local spill files are merged and sorted to create a single partitioned and sorted output file on the local node.

4.2 Reduce Side Execution Details

1. **Copy Phase:** The Reducer tasks have copier threads that fetch their respective partition files from all mapper nodes over HTTP (using the `MapOutputServlet`).
2. **Merge Phase:** As files are copied, background threads merge and sort the incoming data in passes.
3. **Reduce Phase:** The sorted key-value pairs are fed to the `'reduce()'` method line-by-line.

5. MapReduce Types and Formats

MapReduce enforces strict type checking using serialization wrappers (e.g., 'Text' for Java String, 'IntWritable' for Integer).

5.1 Input Splits and InputFormat

- **InputSplit:** A chunk of HDFS data processed by a single mapper. Typically equals the HDFS block size (128MB).
- **InputFormat:** Validates the input files and defines how to split them. It spawns the **RecordReader** to convert split byte streams into key-value records.

InputFormat	Key Type	Value Type
'TextInputFormat'	'LongWritable' (Byte offset of line)	'Text' (Content of the text line)
'KeyValueTextInputFormat'	'Text' (Text before separator)	'Text' (Text after separator)
'SequenceFileInputFormat'	User-defined binary key type	User-defined binary value type

6. Advanced MapReduce Features

6.1 Counters

Counters are used to collect statistics about MapReduce jobs (e.g., counting empty records or corrupt lines).

- **Built-in Counters:** Automatically populated by YARN (e.g., Map Input Records, Bytes Written).
- **User-Defined Counters:** Programmed in Java using enums:
`context.getCounter(MyEnum.CORRUPT_ROWS).increment(1);`

6.2 Joins in MapReduce

When combining datasets (e.g., Customers and Orders tables), two join strategies are used:

Feature	Map-Side Join	Reduce-Side Join
Execution Phase	Done completely inside the Mapper.	Done inside the Reducer.
Prerequisites	Datasets must be sorted and bucketed by the join key. One dataset must be small enough to fit in memory.	None. General-purpose join.
Shuffle Overhead	Zero shuffle overhead (fastest).	Heavy network shuffle of all records.
Data Volume Limit	Limited by Mapper node RAM.	Unlimited (handles massive datasets).

Quick Revision — Module 3

MapReduce Internals, YARN Scheduler & Joins | OE-EC803B

One-Line Definitions

- **ApplicationMaster:** A per-application YARN process that negotiates resources and coordinates task runs.
- **Container:** A logical YARN resource allocation containing CPU cores and RAM limits on a NodeManager.
- **Speculative Execution:** A task execution strategy that runs duplicate copies of slow-running tasks in parallel to minimize job latency.
- **Combiner:** A local semi-reducer that aggregates Map outputs locally before the network shuffle.
- **RecordReader:** A helper class that translates raw HDFS data splits into structured key-value objects.

Frequently Asked / Viva Questions

Q1. What are the differences between MapReduce 1 (Classic) and YARN?

In MapReduce 1, the JobTracker was responsible for both resource management and task scheduling, creating scalability bottlenecks. YARN decouples these tasks, using a global **ResourceManager** for cluster resource allocation and a per-job **ApplicationMaster** for scheduling.

Q2. Detail the spilling process on the Map side.

Mappers write outputs to a 100MB circular buffer. When the buffer is 80% full, a background thread locks the data, partitions it based on Reducer IDs, sorts it by key, runs the optional combiner, and writes (spills) the sorted partition files to local disk.

Q3. How does the Fair Scheduler distribute cluster resources?

The Fair Scheduler divides resources dynamically so that all active jobs receive an equal share of CPU and RAM over time. If only one job runs, it uses the entire cluster. When a second job starts, YARN re-allocates 50% of the resources to it.

Q4. Why does a NodeManager failure force mappers to rerun, but not reducers?

Mappers store their intermediate partition outputs on local disks, so if the mapper's node dies, those files are lost. Reducers write their final outputs directly to replicated HDFS blocks, which remain accessible even if a NodeManager fails.

Q5. What is the difference between TextInputFormat and KeyValueTextInputFormat?

'TextInputFormat' treats the line offset as the key ('LongWritable') and the line text

Frequently Asked / Viva Questions (contd.)

as the value ('Text'). 'KeyValueTextInputFormat' splits each line at a separator character (like a tab) to define both the key and the value as 'Text'.

Q6. Explain Map-side Joins and their memory constraints.

A Map-side Join joins datasets inside the Mapper without a network shuffle. It requires that all datasets are pre-sorted and bucketed by the join key, and one of the datasets must be small enough to load entirely into the mapper's JVM memory.

Q7. Why does speculative execution not run for tasks with side effects?

Speculative execution launches duplicate tasks. If a task writes to an external database, running duplicate tasks would cause duplicate writes. Therefore, speculative execution should be disabled for tasks with external side effects.

Q8. What is the purpose of sync markers in a SequenceFile?

Sync markers are special byte sequences placed periodically in a SequenceFile. They allow MapReduce to split the file at arbitrary boundaries, as the RecordReader can seek to a random position and scan forward to find the next sync marker.

Q9. How does secondary sorting work in MapReduce?

It allows sorting the values associated with a key. It is implemented by creating a composite key (comprising the natural key and the secondary sort value), partitioning by the natural key, and sorting by the composite key.

Q10. What does the HashPartitioner do?

It computes the partition index for a mapper output key: `(key.hashCode() & Integer.MAX_VALUE) % numReducers`. This ensures that all records with the same key are routed to the same Reducer ID.

Comparison Snapshot**Capacity Scheduler vs. Fair Scheduler:**

Feature	Capacity Scheduler	Fair Scheduler
Primary Design Goal	Organization queues with guaranteed shares	Dynamic, equal sharing of cluster resources
Queue Configuration	Strict hierarchical queues (e.g., Sales, Dev)	Simple resource shares per running application
Resource Loaning	Queues can borrow unused capacity from others	Idle capacity is divided among active jobs
Preemption	Not supported (waits for tasks to end)	Supported (reclaims resources by killing tasks)
Use Case	Multi-tenant corporate cluster with fixed budgets	Ad-hoc environments with variable workloads